```
// Headless verification of Demo 30's gauge (Theorem 5.3 kernel)
const Zv=150, Zh=250, D=Zh-Zv;
const toScreen=(P)=>[-P[0]*Zv/(P[2]-Zv), -P[1]*Zh/(P[2]-Zh)];
const address=(P)=>[ P[1]*(Zv-Zh)/(P[2]-Zh), P[0]*(Zh-Zv)/(P[2]-Zv) ];
function gauge(P,g){
  const b=(Zh-P[2])/D, d=(P[2]-Zv)/D, W=g*b+d;
  return [P[0]/W, g*P[1]/W, (g*Zv*(Zh-P[2])+Zh*(P[2]-Zv))/(D*W)];
}
const rnd=()=> (Math.random()-0.5)*200;

// 1) slits fixed pointwise
let w=0;
for(let i=0;i<2000;i++){
  const g=0.6+Math.random()*1.2;
  const q1=gauge([0, rnd(), Zv], g), q2=gauge([rnd(), 0, Zh], g);
  w=Math.max(w, Math.abs(q1[0]), Math.abs(q1[1]-q1[1]), Math.abs(q1[2]-Zv),
             Math.abs(q2[1]), Math.abs(q2[2]-Zh));
}
console.log("slits fixed pointwise, worst    :", w.toExponential(2));

// 2) every ray preserved: address invariant; hence 3) image frozen
let wa=0, wi=0, wdz=0;
for(let i=0;i<20000;i++){
  const g=0.75+Math.random()*0.55;
  const P=[rnd(), rnd(), 300+Math.random()*60];
  const Q=gauge(P,g);
  const a0=address(P), a1=address(Q);
  wa=Math.max(wa, Math.hypot(a0[0]-a1[0], a0[1]-a1[1]));
  const s0=toScreen(P), s1=toScreen(Q);
  wi=Math.max(wi, Math.hypot(s0[0]-s1[0], s0[1]-s1[1]));
  wdz=Math.max(wdz, Math.abs(Q[2]-P[2]));
}
console.log("address drift (rays preserved)  :", wa.toExponential(2));
console.log("image drift (frozen)            :", wi.toExponential(2));
console.log("max |dz| moved (depth NOT froze):", wdz.toFixed(1));

// 4) identity and one-parameter group law: gauge(g1) o gauge(g2) = gauge(g1*g2)
let wg=0;
for(let i=0;i<5000;i++){
  const g1=0.8+Math.random()*0.5, g2=0.8+Math.random()*0.5;
  const P=[rnd(), rnd(), 300+Math.random()*60];
  const a=gauge(gauge(P,g2),g1), b=gauge(P,g1*g2);
  wg=Math.max(wg, Math.hypot(a[0]-b[0],a[1]-b[1],a[2]-b[2]));
  const I=gauge(P,1);
```

```
  wg=Math.max(wg, Math.hypot(I[0]-P[0],I[1]-P[1],I[2]-P[2]));
}
console.log("group law + identity, worst     :", wg.toExponential(2));

// 5) the price of depth: a frontal circle stops being round
const Pc=[-40, 12, 350];
function roundness(g){
  const Gc=gauge(Pc,g); let mn=1e18,mx=0;
  for(let k=0;k<64;k++){
    const th=k/64*2*Math.PI;
    const p=gauge([Pc[0]+16*Math.cos(th), Pc[1]+16*Math.sin(th), Pc[2]], g);
    const r=Math.hypot(p[0]-Gc[0],p[1]-Gc[1],p[2]-Gc[2]);
    mn=Math.min(mn,r); mx=Math.max(mx,r);
  }
  return mx/mn;
}
console.log("roundness at g=1.0 (=1)         :", roundness(1).toFixed(6));
console.log("roundness at g=1.3 (>1)         :", roundness(1.3).toFixed(4));
console.log("dz of that circle's center      :", (gauge(Pc,1.3)[2]-Pc[2]).toFixed
```